

Maithresh Vaddi

AI / ML Engineer · Generative AI · Agentic AI Systems

Hyderabad, Telangana, India | +91 70752 98432 | maithreshvaddi16@gmail.com
maithreshvaddi-27.github.io/maithresh.sh | linkedin.com/in/maithreshvaddi | github.com/MaithreshVaddi-27
leetcode.com/u/MaithreshV | HackerRank

Summary

B.Tech CSE undergraduate (KMIT Hyderabad, graduating 2027, CGPA 7.96/10) pursuing AI/ML Engineering roles, with applied experience across GenAI/LLM application development, agentic AI systems, hybrid retrieval pipelines, ML model integration, and workflow automation. Built TrustRAG, a local-first AI reliability workbench that decomposes RAG answers into verifiable claims, runs NLI-based hallucination checks, self-heals low-confidence responses via a budget-aware LangGraph recovery loop, and runs entirely offline by default (Ollama/llama.cpp/MLX + ONNX embeddings, no API keys required) – backed by a 128-test automated suite plus SAST/container-vulnerability CI gates; specified end-to-end and built via directed AI-orchestrated engineering (Claude Code, OpenCode, AntigraVity). Also solo-built DocuChat, an agentic RAG assistant (CLI plus a full Gradio web app) with local-LLM inference, persistent memory, and provider-agnostic MCP tool routing; CareerOS-Pro, a multi-source async job-aggregation and career-page scraping platform (5 job-board APIs plus a custom scraper) with LangGraph-routed multi-provider LLM fallback; a CrewAI resume-to-job-description matching tool with a CLI and web UI; five MCP-orchestrated task agents (document Q&A, job search, salary research, inbox automation, course curation); a CrewAI multi-agent game-development pipeline; an n8n-orchestrated automation suite of 10 workflows (including agentic builds with voice input, Redis memory, and autonomous Google Doc/Calendar creation); and an RPA email-reminder bot. Contributed the ML-integration and frontend layers on two group AI platforms. Broad, current exposure to the LLM tooling ecosystem – LangChain, LangGraph, MCP, vector databases – through coursework and independent builds.

Technical Skills

Languages: Python, Java (OOP coursework), C++, JavaScript (ES6), SQL

LLMs & GenAI: Google Gemini API, NVIDIA NIM, LangChain, LangGraph (stateful agents, self-healing recovery loops, multi-provider LLM routing), CrewAI, Retrieval-Augmented Generation (RAG), hybrid dense + sparse (BM25) retrieval, Reciprocal Rank Fusion, cross-encoder reranking, NLI-based claim verification, Model Context Protocol (MCP – server and client, JSON-RPC 2.0, Composio orchestration), AI Agents (ReAct / Plan-Execute), conversational memory / checkpointing (InMemorySaver), vector databases (ChromaDB, Qdrant), ONNX Runtime (embeddings + reranking), HuggingFace Embeddings (BAAI/bge-small-en-v1.5, BAAI/bge-base-en-v1.5, all-mpnet-base-v2), prompt engineering, local LLMs (Ollama, llama.cpp, MLX, LM Studio), LoRA/RLHF fundamentals

AI-Orchestrated Engineering: directing multi-agent AI coding tools (Claude Code, OpenCode, AntigraVity) from detailed, self-written specifications to ship production systems faster – full architectural and design ownership retained, implementation delegated deliberately

Automation: n8n, Make.com, Automation Anywhere (RPA), Webhooks, REST APIs

AI / ML: TensorFlow, Keras, Scikit-learn, YOLO, CNN/RNN/LSTM (academic coursework)

Backend & DB: FastAPI, Granian, Flask, httpx (async), Pydantic, SQLAlchemy, Node.js, Express.js, MongoDB, MySQL, SQLite

Cloud & DevOps: AWS – EC2, EFS, EBS, VPC, S3, Lambda, SNS, SQS, Elastic Beanstalk, Lex, IAM, DynamoDB (theory + hands-on labs, no production deployment); Render and Cloudflare Pages (deployed TrustRAG); Docker (Dockerfile, images, docker-compose); Redis; Jenkins, Kubernetes (coursework + self-practice); CI/CD (pytest, ruff, Bandit SAST, Trivy container scanning); Git

Testing & Reliability: pytest, Vitest, Playwright (E2E), k6 (load testing), Bandit (SAST)

Data & Analysis: Pandas, NumPy, Matplotlib; 26 LeetCode SQL problems solved (joins, subqueries, aggregations)

Frontend: React (incl. React 19), HTML5, CSS3, Bootstrap 4.5, Framer Motion, Gradio

Design: Figma (UX research, wireframing)

Projects – AI / GenAI / Agentic Systems

TrustRAG – Local-First RAG Reliability Workbench

Solo – Flagship Project

FastAPI 0.115, React 18/Vite 6, MongoDB 7, Qdrant, LangGraph, ONNX Runtime, MCP (JSON-RPC 2.0)

GitHub

Specified the complete system end-to-end – architecture, the full 8-stage verification pipeline (route → hybrid retrieve → grounded generate → claim decompose → NLI verify → SHA-256 audit → adaptive recover → answer/abstain), data models, and security requirements – then directed implementation via detailed specs across Claude Code, OpenCode, and AntigraVity: a deliberate AI-orchestrated engineering workflow (frontend primarily via Claude Code, backend from my own design), not a substitute for hand-coding but a demonstrated skill in its own right.

Built a local-first architecture that runs entirely offline by default – Ollama, llama.cpp, or MLX for inference, ONNX Runtime (torch-free) for embeddings (BAAI/bge-small-en-v1.5) and int8 cross-encoder reranking (ms-marco-MiniLM-L-6-v2) – with optional cloud fallback to Gemini or NVIDIA NIM; no API keys required for the default path.

Designed claim-level verification: per-claim NLI classification (SUPPORTED / CONTRADICTED / NEUTRAL), SHA-256

evidence-integrity auditing with temporal-validity checks, and a budget-aware LangGraph recovery loop (rewrite → re-retrieve → regenerate, capped at 2 attempts) that safely abstains rather than guessing.

Implemented inference-acceleration and memory-optimization tuning – KV-cache quantization (q4_0/q8_0/fp16), flash attention, prompt caching, speculative decoding, adaptive top-k retrieval, and hierarchical context compression – fully config-driven with environment-variable overrides.

Exposed a JSON-RPC 2.0 MCP server for Claude Desktop/Cursor/Windsurf integration, hybrid dense+BM25 retrieval with Reciprocal Rank Fusion, SSRF-hardened URL validation, JWT auth (HS256, JTI revocation denylist) with bcrypt and login lockout, and production ops tooling (KB snapshots/rollback, Prometheus metrics, A/B experiment framework); backed by 111 backend pytest tests, 15 Vitest tests, 2 Playwright E2E tests, and a k6 load test holding p95 latency under 300ms.

DocuChat – Agentic RAG Assistant with Local LLMs, MCP & Persistent Memory

Solo

Python, LangChain, LangGraph, ChromaDB, Composio (MCP), Gemini, Ollama, llama.cpp, Gradio, SQLite

GitHub

Evolved a CLI-only RAG pipeline (PyPDFLoader ingestion, RecursiveCharacterTextSplitter chunking, BAAI/bge-base-en-v1.5 embeddings, ChromaDB retrieval with stable content-hash IDs) into a Gradio web app supporting three swappable providers – Gemini, Ollama, and llama.cpp (Qwen3-4B-GGUF) – each with automatic readiness checks, while preserving the original CLI (LangGraph checkpointing, retrieve_multi, calculate) as a standalone entry point.

Built provider-agnostic MCP tool routing via Composio (web search by default, extensible to any toolkit with no code change): Gemini always routes through a tool-calling agent, while local providers use the same agent only when tools are configured, otherwise falling back to a faster direct-grounded streaming path.

Added persistent SQLite conversation history, automatic capture of stated personal facts for long-term memory, a size- and age-bounded response cache, per-user rate limiting, and exponential-backoff retries on local-model network hiccups.

Implemented context/prompt/loop "engineering guardrails": every provider receives the same labelled context package (task, memory, conversation, retrieved documents) and a grounding check that triggers an automatic repair pass on ungrounded or citation-less answers.

CareerOS-Pro – Multi-Source Job Aggregation & Career-Page Scraping Platform

Solo

FastAPI/Granian, React 19/Vite/Framer Motion, MySQL, Redis, Qdrant, LangGraph, Docker

GitHub

Built a full-stack job-search platform aggregating live listings from 5 job-board APIs (JSearch, Adzuna, Remotive, RemoteOK, Arbeitnow) plus a custom async, semaphore-controlled company career-page scraper (BeautifulSoup4) with backoff-and-jitter retries; deterministic normalization (no LLM in the parsing path) with two-stage deduplication (url_hash, content_hash) and hard eligibility filters enforced in code, never overridden by the LLM.

Designed a LangGraph state-machine routing/explanation layer with multi-provider LLM fallback (LlamaCpp → NVIDIA NIM → OpenRouter → Gemini) and two-stage listing verification (HTTP HEAD check, then Firecrawl content scrape) before surfacing results to users.

Shipped Qdrant-backed semantic ranking and a live GET /agents/health endpoint reporting per-agent call counts, failure rate, latency, and status (healthy/degraded/down) across every LLM provider and job-source adapter; containerized with Docker/docker-compose.

Resume Crew – Local-First Resume-to-JD Match & Tailoring Tool

Solo

Python, CrewAI, Gradio, Ollama, Gemini API, pypdf, python-docx, pytest

GitHub

Built a CrewAI multi-agent pipeline that parses resume/JD pairs (PDF, DOCX, TXT, Markdown) and produces a six-part evidence-based report – match score, resume/JD profiles, skills-gap analysis, tailored bullet suggestions, and interview prep – combining deterministic keyword scoring with LLM analysis, exported as Markdown, PDF, and Word.

Shipped a Gradio web UI with 8 workflows (Analyze, Build Resume from source facts only, Rank Resumes, Batch Analyze, Compare JDs, History with a score-trend chart, Hardware diagnostics) alongside a scriptable CLI, both supporting local Ollama inference (privacy-first, CUDA/MPS auto-detection) with optional Gemini cloud fallback and password-protected ngrok public sharing.

Covered scoring logic, document validation, report structure, and edge cases (CRLF handling, single-character tokens, timestamp formatting) with a pytest suite spanning six modules; disabled CrewAI tracing/telemetry by default to keep candidate data out of third-party services.

AI Game Development Crew – Multi-Agent CrewAI Pipeline

Solo

Python, CrewAI, Gemini API, Pygame, SerperDevTool, pygbag, ngrok

GitHub

Architected a 3-agent CrewAI pipeline (Game Designer → Senior Python/Pygame Developer → QA Reviewer) with sequential task chaining and shared context, turning a one-line game idea into a complete, playable 2D game.

Equipped the developer agent with live web search (SerperDevTool) for implementation research, then packaged the reviewed game as browser-playable WebAssembly (pygbag) served through a public ngrok tunnel.

Fixed a self-destructive output-path collision where generated game code would have overwritten the launcher script itself, and patched a pygbag/Pyodide WASM compatibility gap by requiring an async-yielding game loop, preventing browser builds from freezing or hanging on load.

MCP Agents Suite – SkillMap & SalaryInsights

Solo

Python, LangChain, Composio (MCP: Tavily, Firecrawl), JSearch API, Gemini

GitHub

SkillMap Agent: chained Composio's MCP-hosted Tavily tools (via MultiServerMCPClient) with a custom search_jobs tool calling the JSearch API; persisted session state (LangGraph in-memory checkpointing) so follow-up queries resolve without restating context; diagnosed and pinned a transitive dependency conflict (mcp>=2.0.0 broke langchain-mcp-adapters imports) to keep the

build reproducible.

SalaryInsights Agent: orchestrated Firecrawl MCP search-and-scrape tools to pull live compensation data from Glassdoor, AmbitionBox, PayScale, and Levels.fyi; system prompt instructs the agent to ground every answer in scraped data with source URLs, with a fallback path if it skips the tool calls.

Email Inbox Summarizer Agent – Gmail Triage & Auto-Draft

Python, LangChain, Composio (MCP: Gmail), Gemini

Solo

GitHub

Wired up Gmail MCP tools (fetch, label, reply, draft) to summarize the unread inbox and classify each item as URGENT / NEEDS REPLY / FYI.

Generated contextual draft replies for flagged emails via GMAIL_CREATE_DRAFT – drafts land in Gmail's Drafts folder for manual review; the system prompt never authorizes auto-send.

AI Blog Writing Crew – Topic-to-Post CrewAI Pipeline

Python, CrewAI, Gemini API

Solo

GitHub

Built a single-file CrewAI pipeline (Content Strategist → Blog Writer → Editor, sequential process) that turns one topic into a polished, timestamped Markdown post; CLI supports topic input, custom output path, and Gemini model override.

CourseFinder Agent – Learning Roadmap & Course Curation

Python, LangChain, Composio (MCP: Tavily, YouTube), Gemini

Solo

GitHub

Merged Tavily MCP (roadmaps, docs, certifications) and YouTube MCP (tutorial/playlist search) results into one structured learning path per topic, with a no-hallucination system prompt grounding every recommendation in tool-call output.

CrimeSleuth – AI-Powered Criminal Investigation Platform

React/TS, Flask, MongoDB, YOLO, Gemini API

Team of 4

GitHub

Trained a 14-class crime-scene classification model on Google Colab, exported as .pth, and loaded it via PyTorch for inference in the deployed Flask app – genuine model training, not a pretrained-model integration; integrated YOLO object detection on evidence images.

Combined classification and detection outputs into Gemini API prompts to auto-generate investigation reports.

SignatureSense – Handwritten Signature Verification

MERN, Flask, TensorFlow/Keras

Team of 4

GitHub

Integrated a pre-trained Keras/TensorFlow model (.h5, loaded via `tf.keras.models.load_model()`) for signature-verification inference.

Owned image preprocessing, integration testing, and Bootstrap frontend fixes.

Automation & Workflow Engineering

n8n Workflow Automation Suite – 10 Workflows

n8n, Gemini, Groq (Whisper), SerpApi, ScraperAPI, OpenWeatherMap, Redis, Telegram Bot API

Solo

GitHub

Shopping Assistant (agentic): Telegram trigger accepting text or voice – Groq Whisper transcribes voice notes – routed to a Gemini tool-using agent with an HTTP tool hitting a scraper API for live product search, backed by per-chat conversational memory via a Redis Chat Memory node.

Learning Path Generator Agent (agentic): chat-triggered Gemini agent with a SerpApi search tool builds a day-wise learning curriculum, writes it to a new Google Doc, and creates matching Google Calendar study-block events.

Internship Applier: Google Sheets row trigger → Gemini drafts a personalized application email → sent via Gmail.

LinkedIn Job Tracker: daily schedule + RSS feed → Gemini extracts required skills and drafts a cover letter → appended as a new Sheet row.

News Summarizer: daily schedule + RSS feeds + SerpApi event fetch, merged and summarized by Gemini into a categorized digest → sent via Gmail.

Weather Daily Planner: daily schedule + OpenWeather API → Gemini writes a day plan around the forecast → sent via Gmail.

Historical Content Publisher: daily schedule → Gemini finds a relevant historical event and drafts a post → published to LinkedIn.

Learning Journey Showcase & Social Media Automation (n8n port): Sheets-triggered (learning log / article links) → Gemini summarizes with parallel platform-specific rewrites → posted to LinkedIn and X; the n8n counterpart to the existing Make.com social pipeline below.

AI Podcast Generator – End-to-End n8n Content Automation

n8n, Gemini API, Murf AI, Lovable

Solo

Live Demo | GitHub

Automated an end-to-end n8n workflow connecting a Lovable-generated frontend to a webhook-triggered podcast pipeline.

Configured Gemini for script generation and Murf AI for text-to-speech, automating audio retrieval and webhook response.

Make.com Automation Suite

Make.com, Gemini AI, Google Sheets, Telegram

Solo

GitHub

Social Media Automation: Google Sheets trigger → AI summarizer → router splits to three platform-personalizer branches → posts to LinkedIn, Facebook Pages, and Telegram; runs on a 15-minute schedule.

Resume Evaluator: a tool-using Make AI Agent triggered via Telegram message, with tool scenarios to fetch resume content and reply by email and Telegram.

RPA Bot – Automated Email Reminder System

Automation Anywhere, Excel

Solo
GitHub

Developed a bot that reads Excel records in a loop, applies date-diff conditional business logic, sends reminder emails, and updates status with Try-Catch error handling.

Other Projects

F5-TTS Podcast Generator

F5-TTS, Kaggle (GPU)

Solo
GitHub

Ran GPU inference and voice-cloning execution on a Kaggle notebook using the F5-TTS model.

FoodMunch – Restaurant Landing Page

HTML5, CSS3, Bootstrap 4.5

Solo
GitHub

Designed a responsive restaurant landing page using Bootstrap.

Education

Keshav Memorial Institute of Technology (KMIT)

B.Tech, Computer Science and Engineering – CGPA 7.96/10

2023 – 2027
Hyderabad, India

Relevant Coursework: Data Structures & Algorithms, Operating Systems, DBMS, Computer Networks, Software Engineering, Cloud Computing (AWS), Cyber Security

Certifications & Training

- **StudyComrade Workshop:** Data Analytics – EDA and visualization on an Amazon e-commerce dataset (Python, Pandas, Matplotlib)
- **Outskill AI Mastermind Workshop:** Foundations of AI, Building Custom AI Assistants & GPTs, Vibe Coding, Claude Artifacts, AI Image & Video Generation, AI-built Websites